TEAM KNOWLEDGE ENGINEERING

团队知识与 Skill 双资产 AI 工作台方案

面向 40 人业务团队的知识、Skill 与 Agent 运行持续治理方案

方案状态：P0 双资产边界与能力侦察

版本：2026-10-13

核心定义

知识回答 Agent“应该知道什么”；Skill 回答 Agent“应该如何完成一类任务”。本工作台负责让两类资产被创建、治理、装载、运行和持续改进。

CHAPTER 01

方案总览与关键决策

> 本文是方案的唯一决策摘要。它定义产品目标、双资产范围、当前架构和阶段；具体流程、资产、治理、实现与验收见后续文档。

1. 产品定义

这是一个面向 40 人业务团队的**知识与 Skill 双资产 AI 工作台**，不是单独的聊天机器人、向量数据库或企业 Agent 平台。

它持续完成两件事：

- 把资料、会议、复盘和业务规则变成有来源、有人负责、可检索、可复核的知识；
- 把高频、低风险、可重复的业务任务变成有契约、可测试、可授权、可发布、可回滚的 Skill。

```
知识：Agent 应该知道什么
Skill：Agent 应该如何完成一类任务
工作台：让知识和 Skill 被创建、治理、装载、运行和持续改进
```

2. 首期要同时证明两条链

选择一个低风险知识域和一个低风险高频任务，服务 5–8 名真实使用者：

```
知识链：资料 → 候选知识 → 事实/范围确认 → Git/MR → WeKnora → 带来源的检索
Skill 链：任务需求 → Skill 草稿 → 契约/权限/effects → 测试回归 → Git/MR → 非生产 profile 装载

知识版本 + Skill digest + profile
  → DSH Agent 运行
  → 结果/人工纠正/业务结果
  → 知识复核或 Skill 回归任务
```

首期限制的是 Skill 的风险和环境，不是取消 Skill 管理。SkillHub 自动发布不可用时，使用 Git/CI + 管理员发布 + DSH 非生产 profile 完成闭环。

3. 核心决策

决策	当前结论
产品边界	知识资产管理、Skill 资产管理和 Agent 运行反馈
正式源	Git + MR/CI + 人工审核
知识检索	WeKnora，只接收已批准知识
Skill 注册/分发	SkillHub（验证后）或管理员流程
Skill 执行	DSH profile、loader、工具策略和人工确认
首期控制面	模块化单体 + 数据库任务表 + worker
业务体验	业务人员提交、确认、审核、查询、反馈；不直接操作 Git
首期 Skill	一个低风险 Skill，完成创建、契约、测试、审核、非生产装载、运行和反馈
Bot	可选 Agent 协作层，不是资产治理权威
高风险动作	人工 gate、最小权限、真实状态读回、审计和回滚

4. 四条不可破坏的原则

1. **双资产、一套治理底座**：知识和 Skill 都有 owner、版本、来源、审核、权限、反馈和回滚，但两者不能混成一种资产。
2. **一类对象一个权威写入面**：Git 保存知识和 Skill 正式源；WeKnora 做知识索引；SkillHub 做 Skill 分发；DSH 做运行。
3. **状态必须有证据**：请求发出、HTTP 200、模型说完成，都不等于业务成功。
4. **自动化降低成本，不替代责任**：模型起草，脚本门禁，人确认事实、范围、风险和发布责任。

5. 阶段顺序

阶段	要证明的结果	进入条件
P0	双资产模型、关系、样本、角色、schema、provider contract 和运行关联清楚	开始真实集成前
P1	一个知识闭环 + 一个低风险 Skill 闭环可创建、审核、装载和运行	P0 证据齐全
P2	知识和 Skill 的运行反馈、质量报告和复核运营持续运行	P1 有真实使用
P3	SkillHub 自动分发、多 profile、beta/stable 和回滚被真实验证	P1/P2 稳定
P4	只有出现规模/复用瓶颈时才演进 Console/Gateway/MCP/多 provider	触发证据出现
P5	Bot 协作、Plugin 和高风险生产写入	安全、审批和回滚证据齐全

阶段由退出证据推进，不按日历自动升级。

6. 成功标准

MVP 退出时：

- 非程序员不操作 Git 也能创建知识草稿和 Skill 草稿；
- 正式知识有 owner、来源、适用范围、版本和复核日期；
- Skill 有 owner、输入/输出契约、知识依赖、权限/effects、测试和固定版本；
- 知识检索返回来源和版本，Skill 运行能反查 digest、profile 和 Git commit；
- 反馈能分别进入知识复核或 Skill 回归；
- 发布、装载、运行和回滚均有审计和人工接管路径；
- 至少有一个真实任务减少重复劳动或返工。

北极星指标是：**有引用且由固定 Skill 可靠完成的有效任务次数**。

CHAPTER 02

问题、用户与产品范围

1. 业务问题

团队不只是缺少知识，也缺少把知识稳定转成 Agent 能力的机制：

- 资料、会议、复盘和规则散落，内容无法持续维护；
- Agent 能回答问题，却不一定能说明依据、版本和适用范围；
- 同一类任务反复依赖个人经验和临时 Prompt，无法复用；
- Skill 没有输入输出契约、权限边界、测试和版本，运行结果不可控；
- 反馈不能判断应该修知识、修 Skill、修权限还是修流程；
- 一开始建设大平台，复杂度超过团队维护能力。

因此目标是**让业务知识变成可引用资产，让高频任务变成可运行 Skill**。

2. 用户与责任

角色	主要动作	负责什么
贡献者	提交资料、提出任务、确认草稿、反馈结果	输入事实和业务需求
领域 steward	确认知识事实、范围、例外和复核周期	知识正确性
Skill owner	定义任务边界、契约、测试、依赖和预期结果	Skill 行为和质量
审核人	审核跨域、敏感、高风险知识或 Skill	发布责任
平台 owner	维护模板、门禁、profile、权限、发布、故障和 coding agent	平台可靠性
使用者	查询知识、选择 Skill、确认结果、报告问题	使用效果和反馈

早期平台 owner 可以只有 1 人，但每个知识域至少有 steward/备份 steward，每个生产 Skill 必须有 owner/backup owner。

3. 首期核心场景

选择一个“低风险、频繁发生、结果可回放”的任务，例如：

- 研究资料初筛：Skill 检索已批准知识，整理来源和证据卡片；
- 流程规则检查：Skill 根据业务输入和知识规则生成检查结果；
- 项目复盘整理：Skill 把复盘材料整理成知识候选和行动项。

场景选择必须写清输入、知识依赖、Skill 产出、使用者、失败代价、人工 gate 和 fallback。

4. MVP 必须有

知识能力

- 收件箱和知识卡片；
- 来源、证据、适用范围和复核日期；
- Git/CI 变更包、审核和回滚；
- WeKnora 索引、ACL、来源和版本引用；
- 反馈和复核任务。

Skill 能力

- Skill 需求登记和模板化创建；
- input/output schema；
- 知识依赖、工具需求、effects 和风险等级；
- golden cases、回归测试和确定性门禁；
- Git/CI 版本、review、digest 和变更记录；
- 非生产 profile 装载、运行、反馈和回滚；
- SkillHub 不可用时的管理员发布 fallback。

运行能力

- 一次运行同时记录 knowledge version、skill version/digest、profile、run、工具调用和人工确认；
- 结果可以反馈到知识复核或 Skill 回归，不自动修改正式资产。

5. 明确不做

- 不让模型直接发布知识或 Skill；
- 不让 Skill 自行获得工具权限或扩大数据范围；
- 不把 WeKnora 当作 Skill Registry；
- 不把 Bot Memory、群聊结论或运行日志当正式资产；
- 不在首期建设多租户微服务、Agent 市场和高风险生产写入；
- 不以 SkillHub 网页自动化绕过审批；
- 不因为一个 Skill 在一次任务中成功就自动晋升 stable。

6. 价值与退出标准

观察：有效引用率、Skill 任务成功率、人工纠正率、重复提问/整理下降、从草稿到可用版本的耗时、过期率、回滚时间和复核关闭时间。

如果知识或 Skill 不能减少重复劳动、返工或交接成本，应停止扩展，先重新评估场景。

CHAPTER 03

端到端知识工作流

> 本文描述业务如何把知识和任务能力变成可运行资产。数据字段见数据、状态与可追溯性，发布门禁见发布、安全与故障处置。

1. 两条生产线

知识生产线

```
资料/会议/复盘
  → 收件箱
  → Agent 生成候选知识卡片
  → 贡献者确认事实和来源
  → steward 确认范围、例外和敏感等级
  → Git 变更包/MR
  → CI + 审核
  → WeKnora 索引确认
  → 可引用知识
```

Skill 生产线

```
任务需求/重复劳动
  → Skill 需求卡
  → Agent 生成 Skill 草稿
  → owner 确认输入、输出、边界和预期结果
  → 声明知识依赖、工具需求、effects、风险和 profile
  → 契约/安全/golden/regression 检查
  → Git 变更包/MR
  → 审核
  → 非生产 profile 装载
  → 可回放运行
  → beta/stable 或退回修改
```

2. 两条线如何汇合

```
用户任务
  → 选择 profile 和固定 Skill digest
  → Skill 调用 WeKnora 检索已授权知识
  → DSH 执行工具和人工确认
  → 返回结果、知识引用、Skill 版本和运行状态
  → 用户/业务结果反馈
  → 知识复核任务 或 Skill 回归任务
```

Skill 不复制知识正文；运行时必须获得结构化的 `knowledge_id/version/source`。知识版本变化、Skill 契约变化和 profile 变化都应能从一次 run 中识别。

3. 每一步的责任和证据

步骤	责任人	输出	最低证据
收件/需求	贡献者	原始资料或 Skill 需求	来源/任务、捕获人、时间
生成草稿	模型/平台	知识或 Skill 草稿	输入引用、生成 run

步骤	责任人	输出	最低证据
业务确认	贡献者/steward/owner	已确认草稿	差异、确认人、待解决项
确定性验证	脚本/CI	门禁结果	schema、secret、测试和依赖结果
审核	steward/审核人	approved 变更	approval、版本、理由
发布/装载	worker/管理员	published 或 loaded/异常	index job、release、digest、profile
运行	使用者/DSH	结果和运行记录	run、knowledge、skill、tool、人工 gate
反馈	使用者	复核或回归任务	上下文、版本、差异和业务结果

4. 反馈分流

- 知识事实、范围、例外或引用错误 → 知识复核；
- Skill 输入输出、步骤、工具调用或结果质量错误 → Skill 回归/修订；
- 权限、敏感性或 profile 错误 → 权限治理任务；
- 运行基础设施失败 → 平台故障任务；
- 业务需求发生变化 → 新的 Skill/知识需求。

反馈不能直接覆盖正式资产。一次成功运行也不能自动形成知识规则或 stable Skill。

5. 非程序员体验

业务人员只需要理解：**提交、确认、审核、选择、运行、反馈**。分支、commit、MR、pipeline、index job、package digest 和 profile 细节可以查看，但由平台 owner 和 coding agent 维护，不要求业务人员手工操作。

6. 异常分支

来源不足停在 candidate；契约或测试失败停在 validating；审核退回停在 authoring/reviewing；索引或装载失败进入补偿；provider 超时进入 unknown/reconciling；高风险动作没有 approval 则拒绝；所有异常保留人工流程。

CHAPTER 04

知识与 Skill 资产模型

> 知识和 Skill 是两类一等资产：知识描述 Agent 应该知道什么，Skill 描述 Agent 如何完成一类任务。两者共享治理底座，但不能混成一种资产。

1. 双资产关系

```
Knowledge asset                    Skill asset
事实/规则/案例/流程/决策              可重复任务的执行契约和实现
        ↑ source/evidence                  ↓ depends_on
        └──────────────── 一次 Agent run ────────────────┘
                          profile + tools + outcome
```

Skill 可以依赖一个或多个知识域/固定知识版本；它不能复制正式知识正文，也不能自行改变知识的权限和状态。

2. 资产分层

资产	说明	首期
原始证据	说明发生了什么	必须
知识卡片	事实、规则、案例、流程和决策	必须
Skill 需求	说明要重复完成什么任务	必须
Skill 包	契约、实现、测试、依赖和风险声明	必须（低风险）
运行记录	说明一次任务实际发生了什么	必须
反馈/复核任务	说明知识或 Skill 哪里需要改	必须
Plugin/MCP	扩展运行时能力	后续
Bot Memory	运行时上下文和协作记忆	可选，不是正式资产

3. 知识资产

元数据

```
id: knowledge.<domain>.<name>
version: 1.0.0
kind: fact | rule | case | procedure | decision
status: captured | candidate | reviewing | approved | publishing | published | stale | deprecated | rejected
owner: team-owner
steward: user-id
source_refs: []
applies_to: []
not_applies_to: []
last_reviewed_at: timestamp
next_review_at: timestamp
evidence_level: E0 | E1 | E2 | E3
sensitivity: public | internal | confidential | restricted
```

正文

1. 一句话结论；

2. 已确认事实；
3. 依据和来源；
4. 适用条件；
5. 不适用范围和例外；
6. 推断或建议（单独标记）；
7. 反例和未知项；
8. 下一次复核条件。

生命周期

```
captured → candidate → reviewing → approved → publishing → published
```

异常/终态：`index_failed`、`index_unknown`、`stale`、`deprecated`、`rejected`、`manual_required`。

只有真实索引确认才能进入 `published`。证据等级不是模型置信度：E0 是未核验输入，E1 是单一来源和基本确认，E2 是责任人确认且有支持来源/记录，E3 是多场景验证并有边界。

4. Skill 资产

元数据

```
id: skill.<domain>.<name>
version: 0.1.0
status: requested | scaffolded | authoring | validating | reviewing | approved | beta | stable | blocked | withdrawn
| deprecated | retired
owner: team-owner
backup_owner: user-id
purpose: structured task description
knowledge_dependencies:
  - id: knowledge.<domain>.<name>
    version: 3.0.1
input_schema: contract/input.schema.json
output_schema: contract/output.schema.json
effects: contract/effects.yaml
risk_level: L0 | L1 | L2 | L3 | L4 | L5
compatible_profiles: []
package_digest: sha256:...
```

最小 Skill 包

```
skills/<domain>/<name>/
├── SKILL.md
├── skill.yaml
├── contract/input.schema.json
├── contract/output.schema.json
├── contract/effects.yaml
├── tests/golden/
├── tests/regression/
├── dependencies.lock
└── CHANGELOG.md
```

Skill 生命周期

```
requested → scaffolded → authoring → validating → reviewing → approved → beta → stable
                                                  ↘ blocked/rejected
stable → withdrawn/deprecated → retired
```

`approved` 表示资产和审核通过；`beta/stable` 表示可被哪些 profile 装载；不能把 Git 合并、SkillHub 上传或一次运行成功混为同一状态。

5. Skill 必须声明和验证

- 输入和输出 schema；
- 依赖的知识 ID、版本策略和引用要求；
- 工具需求、读写 effects、网络和数据范围；
- 风险等级和人工确认点；
- golden cases、回归集和失败态；
- 依赖、许可证、SBOM 和 package digest；
- 兼容 profile、运行环境和资源限制；
- owner、审核人、变更记录、回滚版本和退役条件。

自然语言说明不能替代机器可读契约；Skill 不能自行获得权限、直接写正式知识或依赖漂移的 `latest`。

6. 版本关系

标识	含义
Git commit	知识或 Skill 源码不可变变更
knowledge version	面向业务的知识修订号
Skill semver	Skill 分发版本
package digest	Skill 内容完整性
index version	WeKnora 索引版本
profile version	DSH 实际装载的资产集合
run	一次实际运行和结果

必须能够追踪：

```
run → profile/version → skill/version/digest → knowledge/version/source → Git commit
```

7. 资产边界

- WeKnora 不是 Skill Registry；
- SkillHub 不是知识审批系统；
- DSH 不是正式源和发布审批系统；
- Bot Memory 不是知识或 Skill 正式版本；
- 反馈是候选/任务，不是正文或代码；
- 一次运行成功不等于知识成立或 Skill stable。

CHAPTER 05

角色、治理与运营机制

1. 三类责任

模型：提取、摘要、分类、起草、发现冲突、生成测试候选
脚本/CI：校验 schema、权限、依赖、secret、回归和差异
人：确认事实、范围、任务边界、风险、批准发布和高风险动作

2. RACI

活动	平台 owner	贡献者	知识 steward	Skill owner	审核人
模板和门禁	A/R	C	C	C	I
提交资料/任务需求	I	R	C	C	I
生成知识/Skill 草稿	A	C	C	C	I
确认知识事实和范围	I	R	A/R	C	C
确认 Skill 契约和预期结果	I	C	C	A/R	C
权限/effects/profile 审核	A/R	I	C	R	A
CI、安全和回归检查	A/R	I	I	C	I
发布知识/Skill	C	I	R	R	A
运行反馈分流	R	C	A（知识）	A（Skill）	C
退役/回滚	R	I	A（知识）	A（Skill）	A

早期一人可以兼任多个角色，但知识责任和 Skill 责任必须分别明确，不能因为同一个人兼任就把两类审核混为一类。

3. 必须人工确认

知识

事实、来源、适用范围、例外、敏感等级、跨域可见性、复核日期和是否足以形成通用规则。

Skill

任务是否值得固化、输入输出是否完整、失败态和例外、知识依赖是否正确、工具/effects 是否必要、风险等级、测试结果、profile 兼容性、beta/stable、撤回和回滚。

共同

任何生产写入、不可逆动作、对外发送、权限扩大和数据范围扩大。

4. 风险等级与 gate

级别	示例	默认控制
L0	只读检索、证据卡片	可自动执行，记录审计
L1	摘要、候选、草稿	可自动执行，标记未验证
L2	隔离环境计算和回放	限资源、网络和数据范围

级别	示例	默认控制
L3	创建任务、修改临时状态	范围校验和可恢复记录
L4	敏感读取、业务写入	人工确认、最小权限、读回比对
L5	生产、不可逆或现场动作	强制 gate、双人审批、急停和回滚

Skill 只能声明能力需求，实际权限由 DSH profile、tool policy、用户、workspace、环境和运行时授权共同决定。

5. Skill 治理门禁

一个 Skill 进入 beta 前必须有：契约、知识依赖、effects/权限声明、golden cases、回归结果、owner、固定版本和非生产 profile。

进入 stable 前还必须有：真实任务证据、失败和人工纠正记录、依赖和 SBOM、digest、运行归因、回滚 pin、下一次复核日期和明确批准。

禁止：模型自动标 stable、相似度自动合并 Skill、Skill 自行增加工具、用一次成功运行替代回归、用 `latest` 代替固定版本。

6. 运营节奏

每次变更

知识 steward 检查事实/范围；Skill owner 检查契约/测试/effects；平台 owner 检查 CI、权限和发布证据。

每周复核队列

分别处理知识错误/过期/引用失效、Skill 失败/纠正率/契约冲突、profile 权限异常、索引/装载失败和人工补偿任务。

每月质量回顾

同时看：有效知识引用率、Skill 任务成功率、人工纠正率、运行成本/延迟、版本回滚、过期率、复核关闭时间和重复劳动下降情况。

7. 反馈分流

反馈	进入的治理任务
事实、范围、引用错误	知识复核/MR
输入输出不匹配	Skill 契约修订
步骤或结果质量错误	Skill 回归/MR
工具越权或敏感性问题	权限/profile 冻结和审计
业务规则变化	知识和依赖 Skill 联合复核
运行基础设施失败	平台故障和补偿

反馈不直接覆盖正式知识或 Skill。试点先选择 5–8 名高频使用者和一个知识域/任务，证明双资产闭环有价值后再扩展。

CHAPTER 06

系统架构与权威边界

1. 当前推荐拓扑

```
贡献者 / steward / Skill owner
            ↓
轻量 Web/API 向导或已验证的 DSH 入口
            ↓
双资产生产与治理模块化单体 + 持久化 worker
  ├─ Knowledge catalog / review / change
  ├─ Skill catalog / contract / test / release
  ├─ operation / job / approval / audit / feedback
  ├─ Git provider
  ├─ WeKnora adapter
  └─ SkillHub adapter（验证后；否则管理员流程）
            ↓
DSH profile / loader / tools / human gate / run
```

第一版只有一个新增控制面，但它同时是一等管理知识和 Skill 的资产控制面，不是只管理知识的服务。异步动作先用数据库任务表、租约、重试、读回和人工补偿。

2. 权威边界

对象	权威来源	负责什么	不负责什么
原始资料/任务需求	收件箱/受控附件	保存输入和捕获信息	直接成为正式资产
正式知识	Git + MR/CI/人工审核	正文、来源、差异、版本、回滚	下游直接修改
知识检索	WeKnora	索引、ACL、查询、引用	决定批准或管理 Skill
正式 Skill	Git + MR/CI/人工审核	源码、契约、测试、依赖、版本	运行时自行改写
Skill 注册/分发	SkillHub（验证后）/管理员记录	package、digest、beta/stable、撤回	决定业务事实是否正确
执行和工具权限	DSH profile/policy	装载、工具、人工确认和运行	改变 canonical source
运行反馈	run/trace/反馈表	归因、质量和候选任务	直接发布资产
资产/任务/审计状态	工作台数据库	catalog、job、approval、补偿和关联	代替 Git/WeKnora/DSH 的专业能力

3. 组件职责

工作台控制面

提供业务人员的统一入口和平台 owner 的控制面：资产目录、知识/Skill 草稿、变更包、审核、任务状态、profile 选择、发布状态、反馈、审计和人工补偿。

它不存放未经治理的正式正文，也不绕过下游真实状态。

Git/CI

保存知识、Skill、schema、测试、metadata、依赖锁定、CODEOWNERS 和变更历史。执行格式、secret、契约、权限冲突、链接、依赖和回归门禁。

WeKnora

只接收批准后的知识，提供权限过滤、检索、来源和版本引用。它不能管理 Skill 的契约、测试、发布或运行权限。

SkillHub

提供 Skill package 的注册、分发、版本和 digest（目标部署验证后）。未验证时由管理员流程保证同样的版本、审批、装载和回滚语义。

DSH

提供 profile、Skill loader、Session、工具调用、人工确认和运行结果。它不拥有知识/Skill 的正式源，也不自动批准发布。

DeepSeek Bot

可选的 DSH 协作层，提供 Bot、Group、Memory 和交接；不能替代资产治理、SkillHub、Git、WeKnora 或企业审批。

4. 当前不引入的层

完整 Integration Gateway、Console Plugin、MCP 高风险工具、多 provider、服务端 Bot Host 和多租户不是 P1 的硬依赖。它们只在两个以上知识域/Skill 稳定使用、第二 provider、人工瓶颈、集中权限审计或模块化单体无法接受地发布时评估。

这不影响知识和 Skill 管理是当前产品核心；延后的是更重的实现层，不是资产能力。

5. 不可混淆的边界

- WeKnora 是知识检索，不是 Skill 管理；
- SkillHub 是 Skill 分发，不是知识事实审核；
- DSH 是执行面，不是资产 canonical source；
- MCP 是调用协议，不是异步控制面；
- Webhook 是事实通知，不是授权；
- HTTP 200 不是业务成功；
- 一个 Agent 的成功回答不是通用规则；
- 一个统一页面不等于系统统一。

CHAPTER 07

数据、状态与可追溯性

1. 资产状态

知识

```
captured → candidate → reviewing → approved → publishing → published
```

异常/终态：`index_failed`、`index_unknown`、`stale`、`deprecated`、`rejected`、`manual_required`。

Skill

```
requested → scaffolded → authoring → validating → reviewing → approved → beta → stable
                                                    ├→ blocked
                                                    └→ rejected
stable → withdrawn/deprecated → retired
```

发布/装载不能混为一谈

- `approved`：源文件、契约和审核通过；
- `published`：知识索引真实确认或 Skill package 分发真实确认；
- `loaded`：DSH profile 实际装载了固定 Skill digest；
- `succeeded`：一次运行有真实结果或人工验证；
- `stable`：Skill 经过明确治理批准，可供指定 profile 使用。

2. 运行和反馈状态

异步运行

```
created → queued → running → waiting_human → succeeded
                                    ├→ failed
                                    ├→ timed_out
                                    ├→ cancelled
                                    └→ unknown → reconciling → succeeded/failed/manual_required
```

反馈复核

```
created → triaged → assigned → in_progress → resolved
                                  ├→ rejected
                                  ├→ deferred
                                  └→ escalated
```

反馈任务必须标明目标：`knowledge_review`、`skill_regression`、`permission_review` 或 `platform_incident`。

3. 最小数据模型

```
workspace
actor
asset
asset_revision
knowledge_dependency
skill_contract
skill_test_case
profile
source_reference
operation
job
approval
release
installation
run
feedback
review_task
audit_event
```

关键字段：`workspace_id`、`actor_id`、`asset_id`、`asset_type`、`source_commit`、`knowledge_version`、`skill_version`、`package_digest`、`profile_version`、`idempotency_key`、`correlation_id`、`state_version`、`last_error_code`。

4. 最小追踪链

```
actor
  → operation
  → job
  → asset revision
  → Git commit/MR/CI
  → release/index job/installation
  → profile
  → run
  → knowledge refs + skill digest + tool calls
  → feedback/review task
```

若引入 DSH/DeepSeek Bot，再追加 `dsh_session_id`、`bot_id`、`room_id`、`exchange_id`。资产版本、知识引用和 Skill digest 必须由系统注入，不能从模型文本猜测。

5. 幂等和恢复

- 所有创建、提交、发布、装载和反馈写操作使用 idempotency key；
- 状态更新带 expected state/state version，旧回调不能覆盖新状态；
- 重试前先读回 provider 的 branch/MR/index/package/profile 状态；
- worker 崩溃由租约扫描恢复；无法确认进入对账；
- 事件落库后再确认接收；
- 删除优先软退役，保留旧版本、digest、审计和回滚 pin。

6. 状态证据

状态	必须证据
knowledge published	index confirmation、document/version/source
skill approved	Git commit、CI、contract/test、approval
skill beta/stable	release、digest、profile compatibility、批准和回滚 pin

状态	必须证据
loaded	DSH read-back 的 profile 和 digest
succeeded	实际结果或人工验证
unknown	对账记录和人工处理状态

业务原文、密钥、Cookie、密码、个人敏感信息和工具敏感参数不能进入 Git、WeKnora 正文、长期 Memory 或 trace。

CHAPTER 08

发布、安全与故障处置

1. 知识发布链

```
知识变更包
  → 服务端校验来源、路径、敏感性和幂等键
  → Git branch/MR
  → CI
  → steward/审核人批准
  → 合并受保护 main
  → WeKnora ingest
  → 真实读回 document/version/ACL
  → knowledge = published
```

2. Skill 发布链

```
Skill 变更包
  → 校验 metadata、schema、依赖、effects 和路径
  → contract/golden/regression/security CI
  → Skill owner + 审核人批准
  → 合并受保护 main
  → 打包并生成 digest
  → SkillHub 发布或管理员发布
  → 目标 profile 安装/装载读回
  → beta/stable 或 blocked
```

SkillHub 不可用时，不取消 Skill 管理：使用 Git package、管理员发布、固定 digest、DSH 非生产 profile 和人工回滚，直到自动分发能力验证完成。

3. CI 门禁

共同门禁

格式、schema、ID、路径、owner、sensitivity、来源、secret、个人敏感信息、重复 canonical key、链接、CODEOWNERS、受保护分支和变更范围。

Skill 门禁

input/output schema、知识依赖、effects 和 profile 权限兼容、依赖锁定、许可证/SBOM、golden/regression、digest 生成和危险权限变更。

CI 判断机器可验证事实，不能代替业务 steward 或 Skill owner 判断事实和任务价值。

4. 审核规则

- 知识结构/格式变更：至少 1 名 steward；
- 知识事实、范围、敏感等级变更：steward + 领域审核人；
- Skill 契约、知识依赖、effects 或 profile 变更：Skill owner + 平台/安全审核；
- 跨域、机密、权限和生产写入：双人审核或额外 owner 批准；
- beta/stable、撤回、退役和回滚：明确责任人和审计理由；
- 合并前重新检查 head SHA、CI 和 approval。

5. 权限和数据保护

Skill 只能声明能力需求，实际权限由用户、workspace、profile、tool policy、环境和运行时授权共同决定。普通 Agent 默认看不到高风险工具；工具调用必须重新校验 schema、ACL、risk 和 approval。

Git 只保存脱敏内容、Skill 源码、测试和引用；WeKnora 按文档、空间、用户和业务域过滤；Secret 通过受控 Secret Manager 提供，不进入 Prompt、Skill、Git、Memory 或 trace。

6. 审计

发布、安装、装载、运行、权限变更、索引、审批、回滚和撤回至少记录：actor、action、resource/type、version、source commit、package digest、profile、request、reason、result、时间和 correlation。

审计必须能关联 Git MR/CI、WeKnora index job、SkillHub release、installation、approval、run 和最终真实状态。

7. 故障处置

- CI 失败：不发布；
- WeKnora 索引失败：保留旧索引，进入补偿；
- SkillHub 不可用：继续使用已固定 package 或管理员发布，禁止漂移；
- DSH 装载失败：profile 标记 blocked/unknown，不报告 Skill 可用；
- provider 超时：unknown，读回或人工对账；
- Webhook 丢失：轮询或人工确认；
- 权限服务不可用：敏感工具拒绝；
- 回滚失败：冻结后续发布并升级人工处理。

8. 回滚

知识

定位错误版本 → 冻结/退役 → Git 回到上一有效版本 → 重新索引 → 读回检索结果 → 审计和复盘。

Skill

定位 Skill/package/profile 版本 → 停止新装载 → 回到旧 digest/profile pin → 运行 smoke/regression → 读回实际装载 → 审计和复盘。

取消 HTTP 请求不代表外部动作已取消；所有取消和超时都必须补查真实状态。

CHAPTER 09

MVP 实施、验证与验收

> 本文是唯一的实施路线和验收入口。Skill 管理从 P0 建模，从 P1 与知识闭环并行交付；后续阶段只扩大分发、规模和风险范围。

1. 交付方式

采用：**一个知识域、一个高频低风险任务、一个 Git provider、一个 worker、一个知识 steward、一个 Skill owner、一个非生产 profile**。

每个阶段留下版本、环境、fixture、运行命令、预期/实际结果、失败动作、owner、回滚方式和 Go/No-Go 决策。

2. P0：双资产边界与能力侦察

交付

- 一个真实但脱敏、可回放的知识样本集和任务样本集；
- 知识 steward、Skill owner、备份责任人和平台 owner；
- Knowledge/Skill schema、状态机、关系和统一 run 追踪；
- fake Git/WeKnora/SkillHub/DSH provider 或 adapter contract；
- Skill input/output、effects、profile 和风险模型；
- 目标版本的 DSH loader/profile、WeKnora 状态、SkillHub API 能力记录；
- 不使用生产 Token 的验证证据。

退出

双资产 schema、非法状态、幂等、权限冲突、契约失败、运行归因、故障路径和人工 fallback 均有测试；未知能力都有 owner、日期和处理方式。

3. P1：双资产 MVP 闭环

知识交付

收件箱、知识卡片、变更包、Git provider、CI/CODEOWNERS、审核、WeKnora ingest/status/search、引用和复核任务。

Skill 交付

Skill 需求卡、模板化创建、input/output schema、知识依赖、effects/权限声明、golden/regression、Git/CI 审核、digest、非生产 profile 装载、可回放运行、反馈和旧版本回滚。

SkillHub 未验证时使用管理员发布，但必须保持 package、digest、profile、approval 和回滚状态可追踪。

P1 退出

- 非程序员不操作 Git 即可创建知识和 Skill 草稿；
- 知识检索结果带 document/version/source/ACL；
- Skill 运行能反查 version/digest/profile/knowledge refs；
- 重复请求不重复创建 MR、release、index 或 installation；
- CI 失败、契约冲突或权限超界不能发布/装载；
- DSH 装载失败、索引失败和 provider 超时有清晰状态和人工接管；
- 至少有一项真实任务使用双资产完成。

4. P2：双资产反馈与运营闭环

交付

运行和反馈关联、反馈分流、知识复核队列、Skill 回归队列、每周复核、每月质量报告、stale/纠正率/失败率/权限异常提醒、备份 owner、退役和回滚流程。

退出

反馈不直接改正式资产；连续 4 周有真实使用和复核记录；至少一个质量指标改善；知识问题能进入知识 MR，Skill 问题能进入 Skill 回归；未知状态可人工对账。

5. P3：Skill 分发与稳定发布

在 P1/P2 稳定后，真实验证 SkillHub 的 package 注册、metadata、beta/stable、digest、DSH profile 安装、revoke、audit、webhook 和回滚。

如果 SkillHub 不具备稳定 API，继续使用管理员发布，不改变 Skill 作为核心资产的定位。

6. P4：控制台/Gateway/MCP 演进

只有在两个以上知识域或 Skill 稳定使用、出现第二 provider、人工状态查询成为瓶颈、需要集中权限/审计，或模块化单体无法接受地测试发布时，才拆 Console Plugin、Integration Gateway、adapter、签名事件和低风险 MCP。

新增层不能改变 Git、WeKnora、SkillHub、DSH 和人工审核的权威边界。

7. P5：Bot、Plugin 与高风险能力

只有在知识和 Skill 已有真实价值、运行归因完整、隔离构建、依赖扫描、审批、二次确认、观测和回滚都成立后，才验证 DeepSeek Bot 协作、Plugin 或生产写入。

8. 统一验收矩阵

ID	验证项	通过标准
V-01	双资产样本和角色	脱敏知识/任务样本、owner/steward/备份齐全
V-02	双资产 schema/状态	合法/非法/重复/并发迁移有测试
V-03	Skill 创建	模板、metadata、契约和知识依赖可生成并审核
V-04	Skill 门禁	schema、effects、权限、依赖、golden/regression 有效
V-05	Git 变更	知识/Skill 路径安全、MR 幂等、CI/审批有效
V-06	WeKnora	来源/版本/ACL 可读回，失败保留旧版
V-07	Skill 发布/装载	digest、profile、安装结果和回滚可读回
V-08	非程序员体验	不操作 Git 可创建、确认、审核和运行
V-09	统一运行归因	run 关联 knowledge version、skill digest、profile、commit
V-10	反馈分流	知识问题进知识复核，Skill 问题进 Skill 回归
V-11	安全审计	无 secret 泄露、越权和不可追踪写入
V-12	故障恢复	超时、丢回调、重复和未知状态可对账
V-13	稳定发布	beta/stable 有批准、真实证据、digest 和旧 pin 回滚
V-14	运营价值	至少一个任务减少重复劳动或返工

详细平台实验见验证材料。

9. Coding agent 交付规范

每次只给 coding agent 一个小任务包：背景、范围、输入输出、禁止事项、失败测试、最小实现、验收命令、证据路径和回滚方式。先做 schema、状态机、fake provider、契约和回归，再做 UI 和真实 provider。

Agent 不拥有生产凭据，不直接 merge `main`，不执行任意 shell/路径，不隐藏未验证能力。完成后必须做独立 code review、安全 review 和范围检查。

10. Go/No-Go

以下任一项不通过，就保持当前阶段：双资产权限隔离、审计完整性、契约和回归、知识/Skill 版本可追踪、未知状态处理、旧版本保留、回滚、人工接管、secret 扫描、非程序员可用性或真实业务价值。

CHAPTER 10

验证材料

验证材料只回答一个问题：外部平台和依赖在目标环境中实际证明了什么。

它们不能定义产品范围、替代方案主线，也不能把“架构上可行”写成“已经集成”。特别要区分：

- **Skill 管理是本项目的核心产品能力；**
- **SkillHub 自动注册、分发和晋升是其中一种待验证实现能力。**

即使 SkillHub 自动化不可用，也必须通过 Git/CI、管理员发布和 DSH 非生产 profile 保证 Skill 管理闭环可运行。

使用规则

每个验证项记录：目标版本、部署环境、测试样本、运行命令、预期结果、实际结果、失败路径、证据位置、owner、日期和下一步。

验证结论分为：

- **已确认**：有源码、官方 API 或可重复运行证据；
- **待验证**：架构上可行，但目标版本或部署环境未证明；
- **人工 fallback**：自动化接口未确认，先由管理员操作；
- **不建议依赖**：当前证据不足或风险不适合首期。

材料

- 平台与 Provider 能力验证：Git、WeKnora、DSH、SkillHub、Skill loader、MCP 和安全实验
- DeepSeek Bot 能力评估：Bot/Group/Memory 的源码事实和可选 PoC

与方案的关系

方案中的阶段和 Go/No-Go 是权威入口；验证材料只提供证据。验证结果改变方案时，必须回到方案总览与关键决策更新决策，而不是在验证文档中暗中改变架构。

CHAPTER 11

平台与 Provider 能力验证

> 本文只记录外部能力的验证问题、实验和结论，不定义产品范围。Skill 管理本身是核心产品能力；本文件只验证 SkillHub、DSH loader/profile 等自动化实现是否可用。

1. 验证分级

- **A：已存在可靠的官方 API、源码或本地文档证据，可进入 PoC。**
- **B：架构上可行，但必须在目标版本和真实部署中验证。**
- **C：前置条件未确认，只做只读、fake provider 或人工 fallback。**
- **D：当前不建议作为正式依赖。**

技术可行不等于已经集成。每个 B/C 项必须产生版本、环境、日期和结论。

2. 当前能力矩阵

能力	当前判断	必须验证
Git branch/commit/MR/CI	A/B	幂等、受保护分支、旧 SHA、Webhook 重放
WeKnora 查询/摄取/检索	A/B	ACL、来源/版本、重复、索引失败和旧版本
DSH 入口/插件	B	目标版本 UI/API 扩展、生命周期和身份传递
DSH Skill loader/profile	B	固定 digest 装载、权限过滤、装载读回和回滚
SkillHub 状态读取	B	版本、digest、owner 和状态是否可读
SkillHub 上传/晋升	C	API、审批、审计、回滚和 webhook；未确认则管理员操作
MCP tools/list/call	A/B	profile 过滤、schema、ACL、审批、超时和取消
OTel/观测出口	B	DSH 事件出口、隐私策略和 run/job/asset 关联
多租户 Gateway	B	身份、密钥、workspace 隔离和跨域拒绝
高风险 Agent 发布	D（默认）	人工 approval、服务端 policy、读回和回滚

3. 首先做的实验

V-01：Git provider contract

使用 fake provider 覆盖 branch、batch commit、MR、pipeline 和状态读取；再在隔离项目验证 protected main、expected base/head SHA、Webhook 签名、重复和乱序事件。

通过条件：重复请求不重复创建资源；所有动作能关联 actor、job、commit、MR 和 pipeline；旧 head SHA 合并被拒绝。

V-02：WeKnora 索引和检索

准备测试知识库和两个 workspace，上传带 canonical key、source commit、version、sensitivity 的脱敏文档，测试重复、ACL、hybrid search、旧版本、退役和索引失败。

通过条件：结果返回 document/version/source；跨 workspace 拒绝；索引失败不覆盖旧版本；`approved` 不被错误标成 `published`。

V-03：DSH 入口和 Skill loader

创建只读测试插件或 API bridge，验证入口注册、profile 传递、固定 Skill digest 装载、权限过滤、重启、升级、卸载、失败清理和回滚。

通过条件：实际装载版本可读回；Skill 声明的 effects 不能突破 profile/tool policy；失败则使用管理员安装或外部 Web/API 向导。

V-04：SkillHub capability discovery

固定目标版本和部署方式，查找官方 REST/CLI/API，测试 upload、metadata、beta/stable、digest、revoke、audit、webhook 和 DSH 非生产 profile 安装。

通过条件：发布结果、digest、安装结果和回滚都可查询；否则锁定管理员 fallback，但保留 Skill 管理闭环。

V-05：Skill contract and regression

用一个低风险 Skill 验证 input/output schema、缺字段、未知字段、超大输入、知识依赖缺失、effects 与 profile 冲突、golden cases、回归失败、digest 变化和旧版本回滚。

通过条件：契约或权限冲突阻止发布/装载；回归失败不能晋升 beta/stable；运行记录包含 Skill version/digest、profile 和知识引用。

V-06：MCP security contract

测试合法/非法/超大参数、无 token、跨 workspace、过期 token、无 approval、幂等、超时、断连和 cancellation；确认高风险工具默认不出现在普通 Agent 的列表中。

通过条件：拒绝有稳定错误码和审计；工具结果与真实 provider 状态一致；日志和 trace 无 token、Cookie 和敏感正文。

4. 通用安全检查

检查	方法	必须结果
Secret 泄露	扫描 Git、日志、trace、错误响应	无 token、Cookie、密钥
跨 workspace	构造资源 ID 和 Webhook	全部拒绝
Webhook 重放	重复 event/旧 timestamp	不重复执行
越权合并/晋升	普通用户调用 merge/promote	拒绝并审计
权限冲突	Skill effects 超出 profile	发布和装载均拒绝
参数注入	工具参数携带外部指令	只按 schema/策略处理
外部响应污染	provider 返回恶意文本	不直接成为系统指令
任意文件上传	脚本、超大文件、路径穿越	拒绝或隔离
审计完整性	反查 actor/job/asset/version/digest	每个写动作可追溯

5. 当前结论

第一版优先 fake provider、一个真实 Git/WeKnora 隔离环境和一个低风险 Skill 的非生产回放。SkillHub 自动发布、DSH UI/loader、MCP 高风险工具和完整 Gateway 都必须先有实验结果，不能因为设计了 adapter 就视为已接通；但这些实现未验证不影响 Skill 管理作为产品核心能力继续交付。

CHAPTER 12

DeepSeek Bot 能力评估

> 本文是外部源码和能力验证材料，不是团队知识工程的核心架构。它只说明 DeepSeek Bot 能做什么、不能做什么，以及何时值得做 PoC。

1. 当前结论

基于已核验的 DeepSeek Bot 预览版本，DeepSeek Bot 是建立在 DeepSeek Harness 插件接口之上的多 Bot 组织、协作、长生命周期 Session 和本地记忆插件。

已确认的能力包括：Bot roster、角色化 Agent、Bot-to-Bot 消息、Group Chat、异步交接、Markdown Memory、Team Memory、定时任务、连接器和本地 usage ledger。

未确认它具备：组织知识索引和版本、Skill 注册/审核/分发、企业级 RBAC、多租户、统一审计、WeKnora/SkillHub 原生适配和 OTel/Phoenix/Langfuse 原生导出。

因此它只能作为 DSH 执行面之上的**可选协作层**，不能替代 Git、WeKnora、SkillHub、审批或企业控制面。

2. 能力边界

DeepSeek Bot 能力	在本项目中的正确归属	结论
Bot roster、Session、群聊和交接	Agent 协作层	可候选 PoC
Bot/Team Memory	运行时记忆/反馈候选	可用，但不能当正式知识
正式知识源文件和评审	Git/CI	不替代
知识索引、ACL、版本引用	WeKnora	需要外部适配
Skill 注册、审核、digest 和晋升	SkillHub + Git/CI	未发现原生能力
工具权限和人工确认	DSH profile/policy	可复用，但需身份映射
统一 trace 和业务反馈	观测/反馈系统	需要适配器

3. Memory 的边界

Bot Memory 是本地运行时上下文，即使有自动 review，也不能证明事实经过领域责任人确认、跨场景验证或适合进入 WeKnora。Team Memory 的写权限也不等于正式知识审批。

Memory、群聊结论和 Bot-to-Bot 结果只能：

- 作为当前运行上下文；
- 生成反馈或候选记录；
- 由人确认后进入 Git MR；
- 经审核和索引确认后成为正式知识。

不得直接写入正式知识空间。

4. 推荐验证 PoC

选择一个低风险、只读、可回放任务，例如研究资料初筛和证据卡片整理：

```
Main Bot
  → Researcher Bot: 检索并列出引用
  → Reviewer Bot: 检查证据、范围和冲突
  → Editor Bot: 生成 Git MR 草稿
  → 领域 owner: 审核并合并
  → WeKnora: 索引批准版本
```

验收不看“群聊是否能完成”，而看：

- 每个 Bot 使用的 Skill 和知识版本是否可追溯；
- 结论能否区分事实、推断和未验证建议；
- Memory 是否严格停留在运行时；
- 失败和人工纠正能否生成候选；
- 是否真的减少交接和重复说明。

5. 进入条件和退出条件

只有在 P1/P2 稳定后，才做 Bot 协作 PoC。若需要多用户共享 roster、企业目录、集中任务队列、HA、限流、SLA、统一审计或服务端多租户，才重新评估服务端化；在此之前按本地 DSH 插件处理。

任何 Secret 都必须使用受控 Secret Manager，不能进入模型请求、工具结果、Git、Memory 或 trace。Bot、Room、Session、Exchange 和工具调用必须能关联到统一 run/correlation 链。